Hackathon Platform – Architecture & Safeguards (Draft)
By: Kishlaya (System Architect)

This builds on the earlier architecture proposal and focuses on what Nikhil highlighted: **RBAC, abuse protection, edge cases, and state machines** so the system is production- grade, not just structurally sound.

1. Roles & RBAC (Access Control)

We'll define clear roles and guard all critical flows and routes.

Roles

- `USER` – normal participant
- `ADMIN` – core organizing team (full control)
- `JUDGE` – invited evaluators (limited to scoring/judging views)

Permissions (high level)

- USER:
  - Can register, update profile, create/join team, pay fees, submit project.
  - Can only view and edit their **own** user data and the team they belong to.
  - Cannot access admin/judge data (no other teams' submissions or scores).
- JUDGE:
  - Can view **only assigned submissions** (per track or per judge assignment).
  - Can submit scores and comments.
  - Cannot change system configuration, payments, or team structures.
- ADMIN:
  - Full access to teams, users, payments, submissions, scores.
  - Can manage judges, assign submissions, change status (SHORTLISTED, WINNER).
  - Can perform bulk operations and exports.

Implementation notes

- `users` table includes a `role` field (`USER`, `ADMIN`, `JUDGE`).
- Next.js server handlers + Supabase RLS ensure:
  - Admin and judge routes are protected on server side, not just hidden in UI.
  - Judge queries are always filtered by assigned submissions only.

2. State Machines & Data Constraints

We treat the platform as a set of **finite state machines (FSMs)** to keep transitions controlled and auditable.

2.1 Participant / Team Lifecycle

User & Team Rules

- A user can belong to **at most one active team** per hackathon.
- Each team has:
  - `team_owner_id` (the leader).
  - `max_members` (configurable per event, e.g. 4).
  - `status`: `REGISTERED`, `TEAM_CREATED`, `PAID`, `SUBMITTED`, `UNDER_REVIEW`, `SHORTLISTED`, `WINNER`.

Team Status FSM

- `REGISTERED` → user created, no team yet.
- `TEAM_CREATED` → team exists, members invited, not paid.
- `PAID` → registration fee completed (team or all members, depending on pricing model).
- `SUBMITTED` → team has submitted project before deadline.
- `UNDER_REVIEW` → submissions locked, judges scoring.
- `SHORTLISTED` → admin marks subset of teams as finalists.
- `WON` / `WINNER` → final winners.

Team Edge Cases

- **Joining multiple teams**:

    - Blocked at DB level (unique constraint on user_id per team/hackathon).
- **Team leader leaving**:
    - Either: require leader to transfer `team_owner_id` before leaving, or disallow leaving if they are sole owner until a new owner is assigned.
- **Max team size**:
    - Enforced at DB and application layer; invites that exceed `max_members` are rejected.
- **Invite expiry**:
    - `invite_token` + `invite_expires_at`.
    - Expired tokens cannot be used; admin can re- issue if needed.

2.2 Payment Status FSM

We introduce a dedicated `payment_status` to handle real- world payment edge cases (not just "paid or not").

Fields (per payment record)

- `payment_status`: `INITIATED`, `SUCCESS`, `FAILED`.
- `razorpay_order_id`, `razorpay_payment_id`, `amount`, `currency` (USD), `attempt_count`, timestamps.

Payment FSM

- User clicks pay → create payment record with `payment_status = INITIATED`.
- Razorpay checkout completes:
    - On verified webhook:
        - `SUCCESS`: update `payment_status = SUCCESS`, tie to user/team, move team to `PAID` if conditions met.
        - `FAILED`: update `payment_status = FAILED` if payment failed/cancelled.
- Edge cases:
    - **Delayed webhooks**:
        - Frontend shows "Payment pending verification" state while server waits for webhook.

- **Duplicate webhooks**:
  - Idempotent processing: if `payment_status` already `SUCCESS`, ignore subsequent webhooks for same `razorpay_payment_id`.
- **Failed payments with retries**:
  - New attempt → new `payment` record with incremented `attempt_count`.
- **Refunds/chargebacks**:
  - Optional: track a separate `refund_status` or `notes` field for admin audit.

2.3 Submission Status & Deadline

Submission States

- `DRAFT` – user saved but not finally submitted (optional).
- `SUBMITTED` – final submission before deadline.
- `LOCKED` – after deadline, submission is read- only.
- `UNDER_REVIEW`, `SHORTLISTED`, `WINNER` tie into team status.

Validation & Constraints

- File uploads:
  - Enforce **max file size** (e.g. 50–100 MB) and **allowed formats** (PDF, PPT/PPTX, ZIP).
  - Backend validates before writing to storage (Supabase Storage).
- Deadline:
  - Backend always checks current time vs configured `submission_deadline`.
  - After deadline:
    - Either: block all changes (`LOCKED`).
    - Or: allow admins to override for specific teams (manual adjustments/late submissions) with audit trail.

3. Abuse Protection & Rate Limiting

We'll harden the platform against bot abuse, spam, and brute-force attempts.

CAPTCHA

- Add CAPTCHA (Cloudflare Turnstile or reCAPTCHA) on:
  - Signup
  - Login (after certain failed attempts)
  - Submission form
- This reduces automated spam signups and fake submissions.

Rate Limiting

- Apply rate limiting on API endpoints:
  - Auth routes (signup/login).
  - Team creation/invites.
  - Submission creation/update.
  - Payment initiations.
- Basic strategy:
  - IP or user-based thresholds: e.g. X requests per minute + burst limit.
  - Return friendly error if exceeded; logs for admin review.

Email Verification

- Enforce verified email for key actions:
  - Creating/joining a team.
  - Initiating payment.
  - Submitting a project.
- This adds friction against disposable emails and bots.

4. Admin Panel Requirements (v1 + v2)

We'll go beyond a basic list and design the admin panel for scale.

Admin Features – v1 (MVP)

- View teams and submissions with list filters:
  - Filter by track, status (`PAID`, `SUBMITTED`, etc.).
- View details of each submission: links, files, team members.
- Enter and edit scores:
  - Criteria: innovation, technical complexity, UI/UX, scalability, comments (1–10).
- See aggregate scores and basic leaderboard by track.

Admin Features – v2 (scale)

- Advanced filters:
  - By track, status, total score range, judge.
- Bulk actions:
  - Mark multiple teams as `SHORTLISTED` or `WINNER`.
  - Send bulk emails (shortlisted/winners/custom).
- CSV export:
  - Teams, submissions, scores for external analysis.
- Judge assignment:
  - Assign judges to tracks or specific submissions.
  - Limit judge views to their assigned items only.

5. Implementation Phasing / Next Steps

Phase 1 – Core flow working

- Implement:
  - Auth + profile (with email verification).
  - Team creation/joining with owner + max size + "one team per user" rule.
  - Razorpay payments with `payment_status` FSM (INITIATED → SUCCESS/FAILED).
  - Basic submission form + deadline lock.
  - Basic admin list view + score entry.

Phase 2 – Hardening & Ops

- Add:
    - CAPTCHA + rate limiting.
    - Judge role + assignment.
    - Advanced admin filters, CSV export.
    - More detailed logging/audit fields for critical transitions (status changes, payments, etc.).

The goal is to get a **working, safe v1 live quickly**, then layer on the advanced admin and ops enhancements as we go.